# TRACEFORGE V2 — PHASE 1

> **Phase:** 1 of 4
> **Goal:** Evidence Ledger + SHA-256 Hasher + Artifact Schema + CLI Skeleton
> **Depends on:** Phase 0 fully complete — all imports verified, repo pushed to GitHub
> **Gate:** Do not start Phase 2 until every step here is committed and tests pass

---

## What Phase 1 builds

Phase 1 builds the foundation that every other module plugs into. No module in Phase 2 can work without this. The four things built here are:

1. `core/ledger.py` — SQLite evidence ledger, append-only, chain-of-custody tracking
2. `core/hasher.py` — SHA-256 hashing of evidence files before analysis touches them
3. `core/artifact.py` — Standardised dataclass every module must return
4. `traceforge/cli.py` — Click-based CLI with all subcommand stubs

When Phase 1 is done you will have a working CLI you can run, a ledger that records evidence with hashes, and a schema that every future module is built against.

---

## Step 1.1 — Build the evidence ledger

**File:** `traceforge/core/ledger.py`

**Why this is built first:** Every other component depends on the ledger. The hasher writes to it. The modules read from it. The CLI uses it to manage cases. Build this first, test it first, trust it completely before anything else.

**Why append-only:** Chain of custody means no evidence record is ever modified or deleted after it is written. If you UPDATE or DELETE a row in the evidence log, the chain of custody is broken. The ledger enforces this at the code level — there is no update or delete method.

**Create the file:**

```
nano ~/traceforge/traceforge/core/ledger.py
```

**Paste this code exactly:**

```
"""
TraceForge v2 — Evidence Ledger
Append-only SQLite ledger for chain-of-custody tracking.
No evidence record is ever modified or deleted after creati
on.
"""

import sqlite3
import os
from datetime import datetime, timezone
from pathlib import Path
from loguru import logger


DB_PATH = Path.home() / "traceforge" / "data" / "traceforg
e.db"


def _get_connection() -> sqlite3.Connection:
    """Return a connection to the TraceForge SQLite databas
e."""
    DB_PATH.parent.mkdir(parents=True, exist_ok=True)
    conn = sqlite3.connect(str(DB_PATH))
    conn.row_factory = sqlite3.Row
    return conn


def init_db() -> None:
    """
    Initialise the database schema.
    Creates tables if they do not exist.
    Safe to call multiple times.
    """
    conn = _get_connection()
```

```
    cursor = conn.cursor()

    # Cases table — one row per investigation
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS cases (
            id          TEXT PRIMARY KEY,
            name        TEXT NOT NULL,
            analyst     TEXT NOT NULL,
            created_at  TEXT NOT NULL,
            notes       TEXT
        )
    """)

    # Evidence log — append only, never updated or deleted
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS evidence_log (
            id              INTEGER PRIMARY KEY AUTOINCREME
NT,
            case_id         TEXT NOT NULL,
            analyst         TEXT NOT NULL,
            file_path       TEXT NOT NULL,
            sha256_hash     TEXT NOT NULL,
            file_size_bytes INTEGER NOT NULL,
            recorded_at     TEXT NOT NULL,
            module          TEXT NOT NULL,
            notes           TEXT,
            FOREIGN KEY (case_id) REFERENCES cases(id)
        )
    """)

    # Artifacts table — stores all module outputs
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS artifacts (
            id              INTEGER PRIMARY KEY AUTOINCREME
NT,
            case_id         TEXT NOT NULL,
            source_module   TEXT NOT NULL,
            artifact_type   TEXT NOT NULL,
```

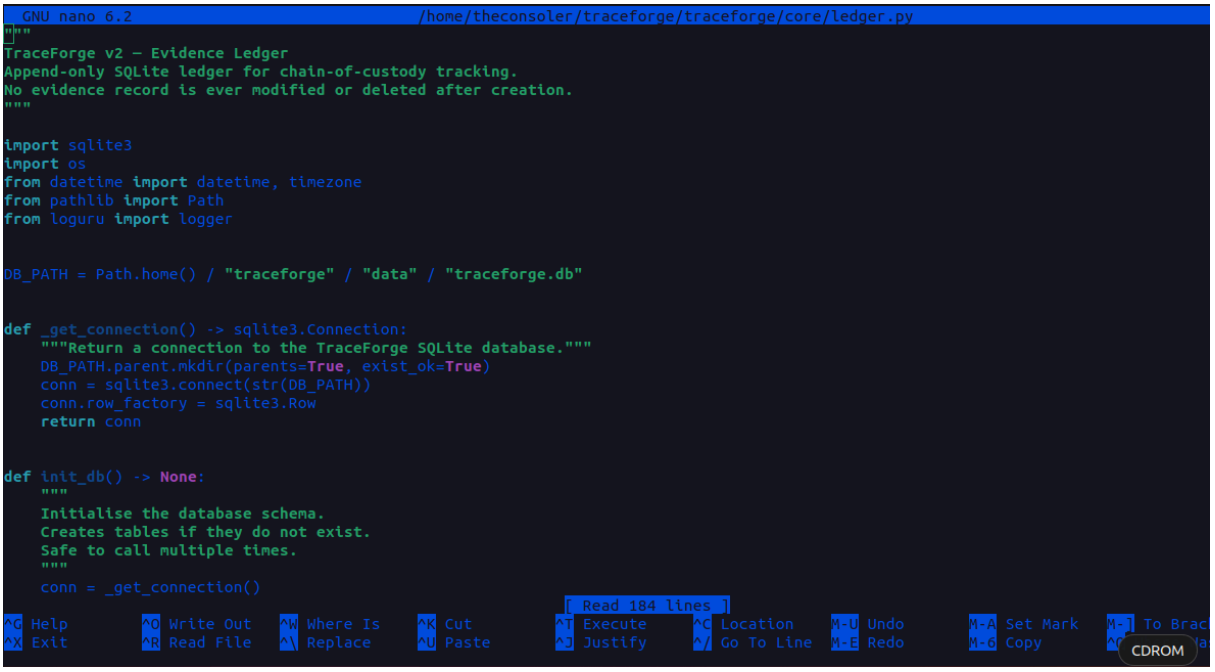
```
            host_id         TEXT,
            timestamp       TEXT,
            data_json       TEXT NOT NULL,
            recorded_at     TEXT NOT NULL,
            FOREIGN KEY (case_id) REFERENCES cases(id)
        )
    """)

    conn.commit()
    conn.close()
    logger.info(f"Database initialised at {DB_PATH}")


def create_case(case_id: str, name: str, analyst: str, notes: str = "") -> dict:
    """
    Create a new investigation case.
    Raises ValueError if case_id already exists.
    """
    conn = _get_connection()
    cursor = conn.cursor()

    existing = cursor.execute(
        "SELECT id FROM cases WHERE id = ?", (case_id,)
    ).fetchone()

    if existing:
        conn.close()
        raise ValueError(f"Case '{case_id}' already exists.")

    created_at = datetime.now(timezone.utc).isoformat()

    cursor.execute(
        "INSERT INTO cases (id, name, analyst, created_at, notes) VALUES (?, ?, ?, ?, ?)",
        (case_id, name, analyst, created_at, notes)
    )
```

```
    conn.commit()
    conn.close()

    logger.info(f"Case created: {case_id} — {name} (analys
t: {analyst})")
    return {
        "case_id": case_id,
        "name": name,
        "analyst": analyst,
        "created_at": created_at,
        "notes": notes
    }


def log_evidence(
    case_id: str,
    analyst: str,
    file_path: str,
    sha256_hash: str,
    file_size_bytes: int,
    module: str,
    notes: str = ""
) -> int:
    """
    Record an evidence file in the append-only ledger.
    The hash must be computed BEFORE this function is calle
d.
    Returns the row ID of the new ledger entry.
    """
    conn = _get_connection()
    cursor = conn.cursor()

    case_exists = cursor.execute(
        "SELECT id FROM cases WHERE id = ?", (case_id,)
    ).fetchone()

    if not case_exists:
```

```
        conn.close()
        raise ValueError(f"Case '{case_id}' does not exist.
Create it first.")

    recorded_at = datetime.now(timezone.utc).isoformat()

    cursor.execute("""
        INSERT INTO evidence_log
            (case_id, analyst, file_path, sha256_hash, file
_size_bytes, recorded_at, module, notes)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)
    """, (case_id, analyst, file_path, sha256_hash, file_si
ze_bytes, recorded_at, module, notes))

    row_id = cursor.lastrowid
    conn.commit()
    conn.close()

    logger.info(f"Evidence logged: {file_path} | SHA256: {s
ha256_hash[:16]}... | Case: {case_id}")
    return row_id


def get_case(case_id: str) -> dict | None:
    """Retrieve a case by ID. Returns None if not found."""
    conn = _get_connection()
    row = conn.execute(
        "SELECT * FROM cases WHERE id = ?", (case_id,)
    ).fetchone()
    conn.close()
    return dict(row) if row else None


def get_evidence_log(case_id: str) -> list[dict]:
    """Return all evidence entries for a case, ordered by r
ecorded_at."""
    conn = _get_connection()
    rows = conn.execute(
```

```
        "SELECT * FROM evidence_log WHERE case_id = ? ORDER
BY recorded_at ASC",
        (case_id,)
    ).fetchall()
    conn.close()
    return [dict(r) for r in rows]


def list_cases() -> list[dict]:
    """Return all cases in the database."""
    conn = _get_connection()
    rows = conn.execute("SELECT * FROM cases ORDER BY creat
ed_at DESC").fetchall()
    conn.close()
    return [dict(r) for r in rows]
```

**Save and exit:** `Ctrl+X` then `Y` then `Enter`




## Step 1.2 — Build the SHA-256 hasher

**File:** `traceforge/core/hasher.py`

**Why hashing comes before analysis:** This is the single most important forensic principle in the entire tool. The hash is computed on the raw, untouched

evidence file and recorded in the ledger. If the file is ever modified during or after analysis, the hash will no longer match and the chain of custody is broken. This is how you prove evidence integrity in court or in a professional IR report.

**Why chunked reading:** Memory dumps and disk images can be 4GB, 8GB, or larger. Reading the entire file into memory to hash it will crash the process. Chunked reading processes 64KB at a time and handles any file size.

```
nano ~/traceforge/traceforge/core/hasher.py
```

```
"""
TraceForge v2 — Evidence Hasher
Computes SHA-256 hash of evidence files before any analysis
begins.
Uses chunked reading to handle large files (memory dumps, d
isk images).
"""

import hashlib
from pathlib import Path
from loguru import logger


CHUNK_SIZE = 65536  # 64KB chunks


def compute_sha256(file_path: str | Path) -> tuple[str, in
t]:
    """
    Compute the SHA-256 hash of a file using chunked readin
g.
    Returns a tuple of (hex_digest, file_size_in_bytes).
    Raises FileNotFoundError if the file does not exist.
    Raises PermissionError if the file cannot be read.
    """
    path = Path(file_path)

    if not path.exists():
```

```
        raise FileNotFoundError(f"Evidence file not found:
{file_path}")

    if not path.is_file():
        raise ValueError(f"Path is not a file: {file_pat
h}")

    sha256 = hashlib.sha256()
    file_size = 0

    logger.info(f"Computing SHA-256 for: {path.name}")

    with open(path, "rb") as f:
        while chunk := f.read(CHUNK_SIZE):
            sha256.update(chunk)
            file_size += len(chunk)

    digest = sha256.hexdigest()
    logger.info(f"SHA-256: {digest} | Size: {file_size:,} b
ytes | File: {path.name}")

    return digest, file_size


def verify_hash(file_path: str | Path, expected_hash: str)
-> bool:
    """
    Verify a file's current SHA-256 hash against an expecte
d value.
    Returns True if the hash matches, False if the file has
been modified.
    Used to verify evidence integrity at any point after in
itial recording.
    """
    path = Path(file_path)

    try:
        current_hash, _ = compute_sha256(path)
```

```
        match = current_hash.lower() == expected_hash.lower
()

        if match:
            logger.info(f"Hash verified OK: {path.name}")
        else:
            logger.warning(
                f"HASH MISMATCH — evidence may be corrupted
or modified: {path.name}"
            )
            logger.warning(f"Expected: {expected_hash}")
            logger.warning(f"Current:  {current_hash}")

        return match

    except FileNotFoundError:
        logger.error(f"Cannot verify — file not found: {fil
e_path}")
        return False
```


## Step 1.3 — Build the artifact schema

**File:** `traceforge/core/artifact.py`

**Why this exists:** Every analysis module (memory, disk, logs, network) produces different kinds of output. Without a standard schema, the correlation engine in Phase 3 has no way to compare artifacts across modules. This dataclass is the contract. Every module must return a list of Artifact objects. No exceptions.

**Why a dataclass:** Clean, typed, serialisable to dict and JSON. No overhead. Every field has a clear purpose.

```
nano ~/traceforge/traceforge/core/artifact.py
```

```
"""
TraceForge v2 — Artifact Schema
Standardised dataclass returned by every analysis module.
The correlation engine depends on this schema — do not chan
ge
field names without updating correlator.py in Phase 3.
"""

import json
from dataclasses import dataclass, field, asdict
from datetime import datetime, timezone
from typing import Any


@dataclass
class Artifact:
    """
    A single forensic artifact produced by any TraceForge m
odule.

    Fields:
        case_id       : The investigation case this artifac
t belongs to
        source_module : Which module produced this (memory/
disk/logs/network)
        artifact_type : What kind of artifact (process/conn
```

```
ection/login/dns_query etc)
        timestamp     : ISO 8601 timestamp of the artifact
event (not when it was recorded)
        host_id       : IP address, hostname, or machine id
entifier
        data          : Dict of module-specific artifact de
tails
        recorded_at   : When this artifact was stored (auto
-set, do not pass manually)
    """
    case_id:       str
    source_module: str
    artifact_type: str
    host_id:       str
    data:          dict[str, Any]
    timestamp:     str = ""
    recorded_at:   str = field(default_factory=lambda: date
time.now(timezone.utc).isoformat())

    # Valid module names — enforced on creation
    VALID_MODULES = {"memory", "disk", "logs", "network"}

    # Common artifact types per module
    ARTIFACT_TYPES = {
        "memory":  {"process", "network_connection", "dll",
"cmdline", "registry_key"},
        "disk":    {"file", "deleted_file", "directory", "m
ft_entry"},
        "logs":    {"failed_login", "successful_login", "su
do_command", "service_event"},
        "network": {"dns_query", "http_request", "tcp_flo
w", "suspicious_connection"},
    }

    def __post_init__(self):
        if self.source_module not in self.VALID_MODULES:
            raise ValueError(
                f"Invalid source_module '{self.source_modul
```

```
e}'. "
                f"Must be one of: {self.VALID_MODULES}"
            )

    def to_dict(self) -> dict:
        """Serialise artifact to a plain dictionary."""
        return asdict(self)

    def to_json(self) -> str:
        """Serialise artifact to a JSON string."""
        return json.dumps(self.to_dict(), indent=2)

    @classmethod
    def from_dict(cls, data: dict) -> "Artifact":
        """Deserialise an artifact from a dictionary."""
        return cls(
            case_id=data["case_id"],
            source_module=data["source_module"],
            artifact_type=data["artifact_type"],
            host_id=data["host_id"],
            data=data["data"],
            timestamp=data.get("timestamp", ""),
            recorded_at=data.get("recorded_at", datetime.no
w(timezone.utc).isoformat())
        )

    def summary(self) -> str:
        """Return a one-line human-readable summary of this
artifact."""
        ts = self.timestamp or self.recorded_at
        return (
            f"[{self.source_module.upper()}] {self.artifact
_type} | "
            f"host={self.host_id} | ts={ts[:19]} | "
            f"data={list(self.data.keys())}"
        )
```


## Step 1.4 — Build the CLI entry point

**File:** `traceforge/cli.py`

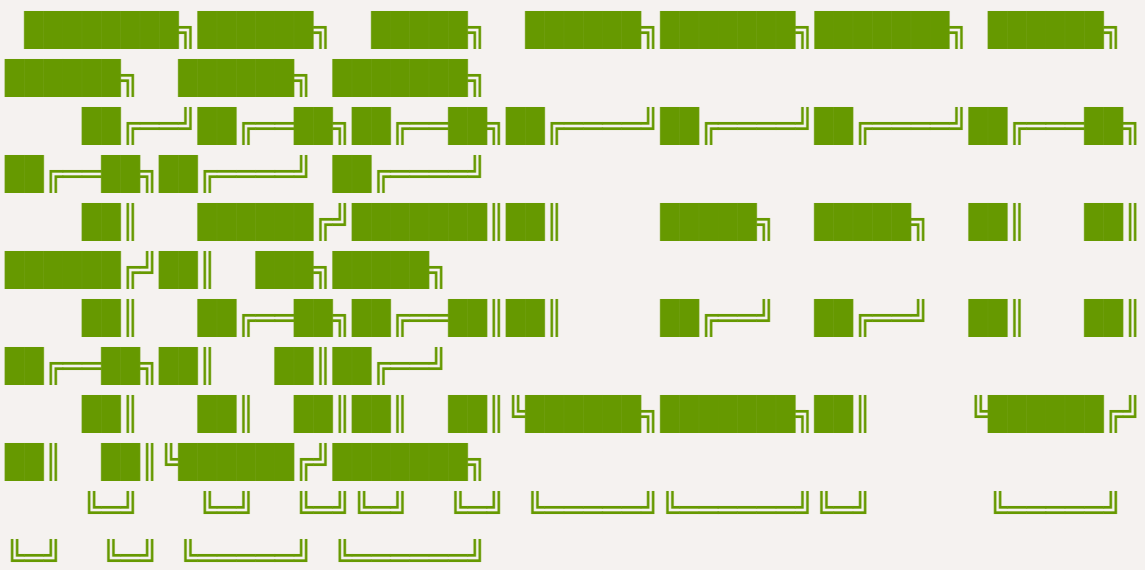
**Why Click and not argparse:** Click produces cleaner subcommand structures, better help text, and handles type validation automatically. The command structure matches how real forensic tools work — subcommands, not interactive menus.

**Why stubs at this stage:** The CLI skeleton connects all the pieces. Each subcommand stub confirms the import chain works before the modules are built. A stub that prints a clear message is infinitely better than a placeholder that silently does nothing.

```
nano ~/traceforge/traceforge/cli.py
```

```
"""
TraceForge v2 — CLI Entry Point
Run: python -m traceforge.cli --help
"""

import click
from rich.console import Console
```

```
from rich.table import Table
from rich import box
from loguru import logger
import sys
import os

from traceforge.core.ledger import init_db, create_case, ge
t_case, list_cases, log_evidence
from traceforge.core.hasher import compute_sha256, verify_h
ash

console = Console()

BANNER = """
[bold cyan]
```



```
[/bold cyan]
[bold white]    Digital Forensics & Incident Response Toolk
it v2[/bold white]
[dim]    github.com/theconsoler/TRACEFORGE-V2[/dim]
"""


@click.group()
def cli():
    """TraceForge v2 — DFIR Toolkit"""
    init_db()
```

```
# ── CASE MANAGEMENT ──────────────────────────────────────────────────

@cli.group()
def case():
    """Create and manage investigation cases."""
    pass


@case.command("new")
@click.option("--id",       "case_id",  required=True,  help="Unique case identifier (e.g. CASE-2026-001)")
@click.option("--name",     required=True,              help="Case name or description")
@click.option("--analyst",  required=True,              help="Analyst name or ID")
@click.option("--notes",    default="",                 help="Optional notes")
def case_new(case_id, name, analyst, notes):
    """Create a new investigation case."""
    try:
        result = create_case(case_id, name, analyst, notes)
        console.print(f"\n[bold green]Case created successfully[/bold green]")
        console.print(f"  ID      : [cyan]{result['case_id']}[/cyan]")
        console.print(f"  Name    : {result['name']}")
        console.print(f"  Analyst : {result['analyst']}")
        console.print(f"  Created : {result['created_at']}\n")
    except ValueError as e:
        console.print(f"[bold red]Error:[/bold red] {e}")
        sys.exit(1)


@case.command("list")
```

```
def case_list():
    """List all investigation cases."""
    cases = list_cases()
    if not cases:
        console.print("[yellow]No cases found. Create one w
ith: traceforge case new[/yellow]")
        return

    table = Table(box=box.ROUNDED, show_header=True, header
_style="bold cyan")
    table.add_column("Case ID", style="cyan")
    table.add_column("Name")
    table.add_column("Analyst")
    table.add_column("Created")

    for c in cases:
        table.add_row(c["id"], c["name"], c["analyst"], c
["created_at"][:19])

    console.print(table)


@case.command("info")
@click.argument("case_id")
def case_info(case_id):
    """Show details for a specific case."""
    c = get_case(case_id)
    if not c:
        console.print(f"[bold red]Case '{case_id}' not foun
d.[/bold red]")
        sys.exit(1)

    console.print(f"\n[bold cyan]Case: {c['id']}[/bold cya
n]")
    console.print(f"  Name    : {c['name']}")
    console.print(f"  Analyst : {c['analyst']}")
    console.print(f"  Created : {c['created_at']}")
    console.print(f"  Notes   : {c['notes'] or 'None'}\n")
```

```
# ── EVIDENCE MANAGEMENT ──────────────────────────────────────────────────────

@cli.command()
@click.option("--case",     "case_id",  required=True, help="Case ID to log evidence for")
@click.option("--file",     "file_path", required=True, help="Path to evidence file")
@click.option("--analyst",  required=True,              help="Analyst name or ID")
@click.option("--module",   required=True,
              type=click.Choice(["memory", "disk", "logs", "network"]),
              help="Module this evidence is for")
@click.option("--notes",    default="",                 help="Optional notes")
def ingest(case_id, file_path, analyst, module, notes):
    """Hash and log an evidence file into the ledger before analysis."""
    console.print(f"\n[bold]Ingesting evidence:[/bold] {file_path}")

    try:
        sha256, size = compute_sha256(file_path)
        row_id = log_evidence(case_id, analyst, file_path, sha256, size, module, notes)

        console.print(f"[bold green]Evidence logged successfully[/bold green]")
        console.print(f"  Ledger ID : {row_id}")
        console.print(f"  SHA-256   : [cyan]{sha256}[/cyan]")
        console.print(f"  Size      : {size:,} bytes")
        console.print(f"  Module    : {module}\n")

    except FileNotFoundError as e:
```

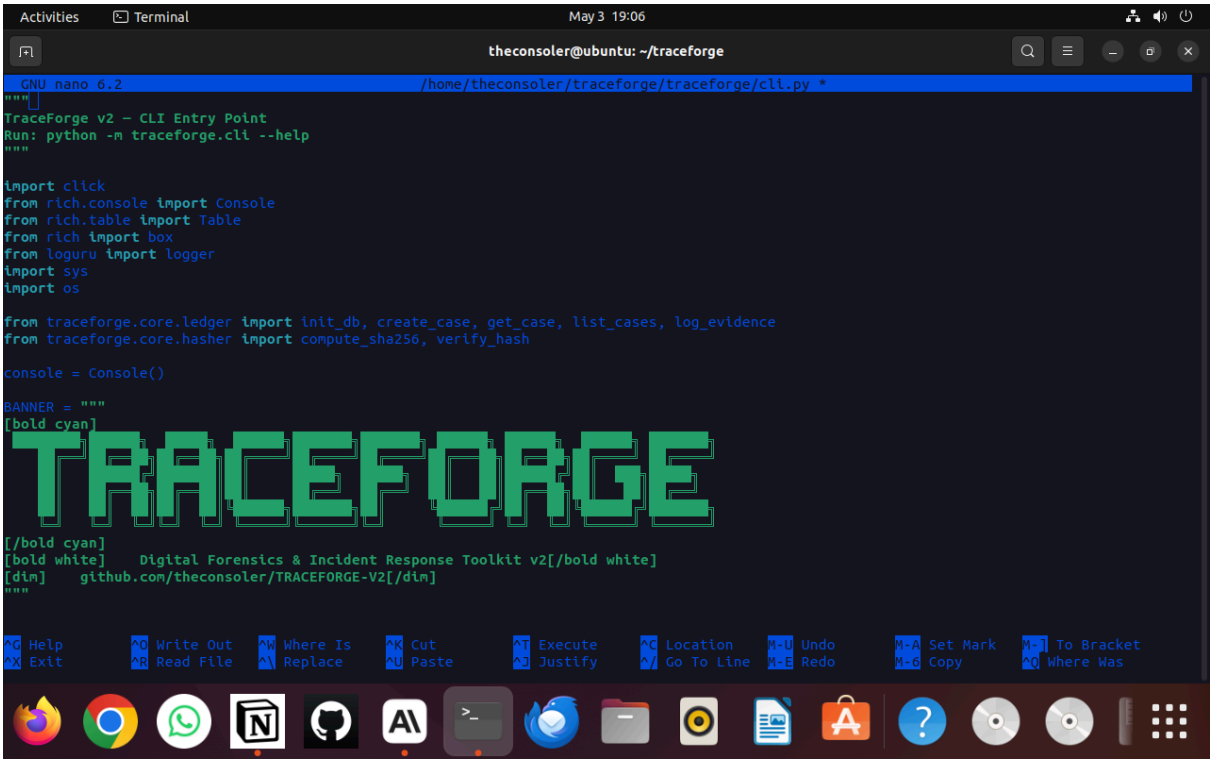
```
        console.print(f"[bold red]Error:[/bold red] {e}")
        sys.exit(1)
    except ValueError as e:
        console.print(f"[bold red]Error:[/bold red] {e}")
        sys.exit(1)


# ── ANALYSIS MODULE STUBS ──────────────────────────────
──────────────────


@cli.command()
@click.option("--case",  "case_id",   required=True, help
="Case ID")
@click.option("--image", required=True,                help
="Path to memory image file")
def memory(case_id, image):
    """Run memory forensics on a memory dump. (Phase 2)"""
    console.print(f"[yellow]Memory module — coming in Phase
2[/yellow]")
    console.print(f"  Case  : {case_id}")
    console.print(f"  Image : {image}")


@cli.command()
@click.option("--case",  "case_id",   required=True, help
="Case ID")
@click.option("--image", required=True,                help
="Path to disk image file")
def disk(case_id, image):
    """Run disk image analysis. (Phase 2)"""
    console.print(f"[yellow]Disk module — coming in Phase 2
[/yellow]")
    console.print(f"  Case  : {case_id}")
    console.print(f"  Image : {image}")


@cli.command()
@click.option("--case",  "case_id",   required=True, help
```

```
="Case ID")
@click.option("--file",  "log_file",  required=True, help
="Path to log file")
def logs(case_id, log_file):
    """Run log correlation analysis. (Phase 2)"""
    console.print(f"[yellow]Logs module — coming in Phase 2
[/yellow]")
    console.print(f"  Case : {case_id}")
    console.print(f"  File : {log_file}")


@cli.command()
@click.option("--case",  "case_id",   required=True, help
="Case ID")
@click.option("--pcap",  required=True,              help
="Path to PCAP file")
def network(case_id, pcap):
    """Run network packet analysis. (Phase 2)"""
    console.print(f"[yellow]Network module — coming in Phas
e 2[/yellow]")
    console.print(f"  Case : {case_id}")
    console.print(f"  PCAP : {pcap}")


@cli.command()
@click.option("--case",   "case_id",  required=True, help
="Case ID")
@click.option("--format", "fmt",      default="json",
              type=click.Choice(["json", "html", "pdf"]),
              help="Output format")
def report(case_id, fmt):
    """Generate case report. (Phase 3)"""
    console.print(f"[yellow]Report module — coming in Phase
3[/yellow]")
    console.print(f"  Case   : {case_id}")
    console.print(f"  Format : {fmt}")
```

```
@cli.command()
@click.option("--case", "case_id", required=True, help="Cas
e ID to correlate")
def correlate(case_id):
    """Run cross-module correlation engine. (Phase 3)"""
    console.print(f"[yellow]Correlation engine — coming in
Phase 3[/yellow]")
    console.print(f"  Case : {case_id}")


@cli.command()
def banner():
    """Show TraceForge banner."""
    console.print(BANNER)


if __name__ == "__main__":
    cli()
```

## Step 1.5 — Create the data directory and update `__init__.py`

```
mkdir -p ~/traceforge/data

cat > ~/traceforge/traceforge/__init__.py << 'EOF'
"""TraceForge v2 — Digital Forensics & Incident Response To
olkit"""
__version__ = "2.0.0"
__author__ = "theconsoler"
EOF
```




## Step 1.6 — Write the Phase 1 tests

**File:** `tests/test_phase1.py`

These tests verify the ledger, hasher, and artifact schema all work correctly before Phase 2 begins.

```
nano ~/traceforge/tests/test_phase1.py
```

```
"""
TraceForge v2 — Phase 1 Tests
Tests for ledger, hasher, and artifact schema.
Run with: pytest tests/test_phase1.py -v
"""

import pytest
import os
import tempfile
from pathlib import Path
from traceforge.core.ledger import init_db, create_case, lo
g_evidence, get_case, get_evidence_log
from traceforge.core.hasher import compute_sha256, verify_h
```

```
ash
from traceforge.core.artifact import Artifact


# ── LEDGER TESTS ──────────────────────────────────────────────
──────────────────────


def test_init_db():
    """Database initialises without errors."""
    init_db()


def test_create_case():
    """A case can be created and retrieved."""
    init_db()
    case = create_case("TEST-001", "Phase 1 Test Case", "th
econsoler")
    assert case["case_id"] == "TEST-001"
    assert case["name"] == "Phase 1 Test Case"
    assert case["analyst"] == "theconsoler"


def test_duplicate_case_raises():
    """Creating a case with an existing ID raises ValueErro
r."""
    init_db()
    try:
        create_case("TEST-DUP", "Duplicate Test", "theconso
ler")
    except ValueError:
        pass
    with pytest.raises(ValueError):
        create_case("TEST-DUP", "Duplicate Test", "theconso
ler")


def test_get_case_not_found():
    """get_case returns None for a non-existent case."""
```

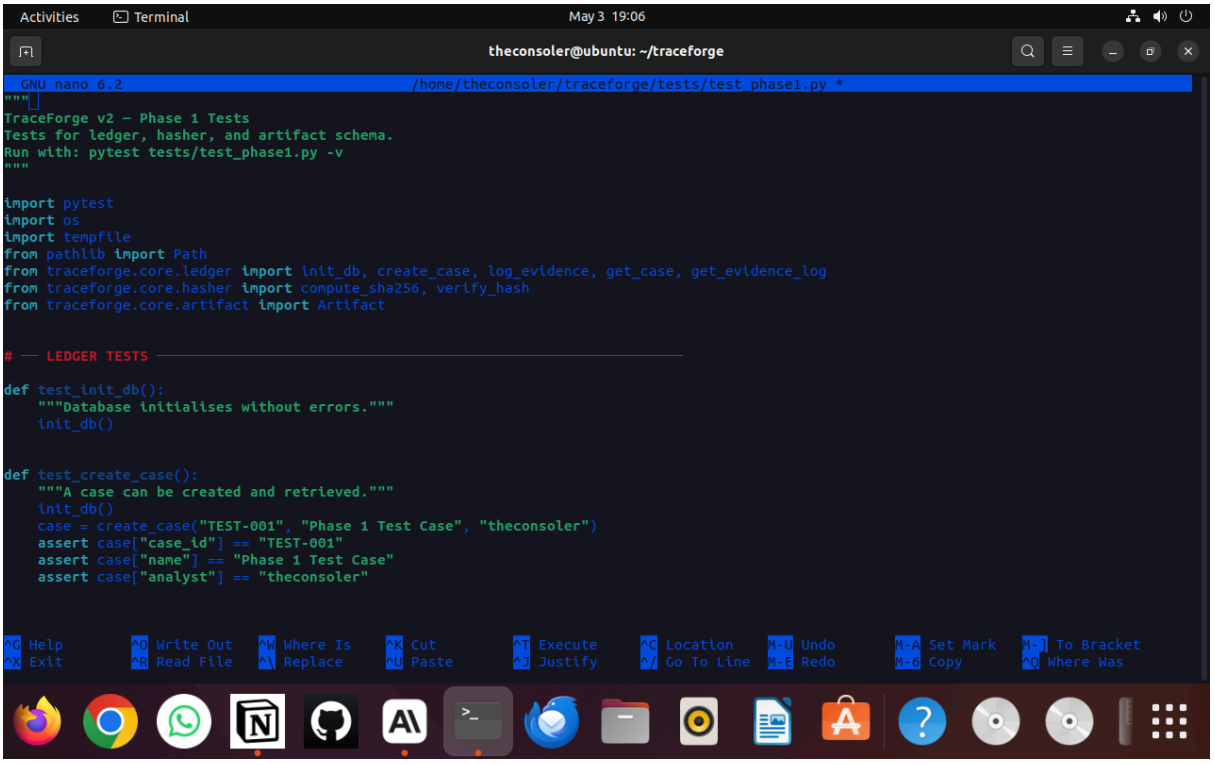
```
    init_db()
    result = get_case("NONEXISTENT-9999")
    assert result is None


def test_log_evidence():
    """Evidence can be logged against a valid case."""
    init_db()
    try:
        create_case("TEST-EVID", "Evidence Test", "theconso
ler")
    except ValueError:
        pass

    with tempfile.NamedTemporaryFile(delete=False, suffix
=".bin") as f:
        f.write(b"fake evidence data for testing" * 100)
        tmp_path = f.name

    try:
        sha256, size = compute_sha256(tmp_path)
        row_id = log_evidence(
            case_id="TEST-EVID",
            analyst="theconsoler",
            file_path=tmp_path,
            sha256_hash=sha256,
            file_size_bytes=size,
            module="memory",
            notes="test evidence"
        )
        assert isinstance(row_id, int)
        assert row_id > 0

        log = get_evidence_log("TEST-EVID")
        assert len(log) > 0
        assert log[-1]["sha256_hash"] == sha256

    finally:
```

```
        os.unlink(tmp_path)


# ── HASHER TESTS ──────────────────────────────────────────────
──────────────────────


def test_sha256_consistency():
    """Same file produces same hash every time."""
    with tempfile.NamedTemporaryFile(delete=False) as f:
        f.write(b"traceforge test content 12345")
        tmp_path = f.name

    try:
        hash1, size1 = compute_sha256(tmp_path)
        hash2, size2 = compute_sha256(tmp_path)
        assert hash1 == hash2
        assert size1 == size2
        assert len(hash1) == 64  # SHA-256 hex digest is al
ways 64 chars
    finally:
        os.unlink(tmp_path)


def test_sha256_file_not_found():
    """FileNotFoundError raised for missing file."""
    with pytest.raises(FileNotFoundError):
        compute_sha256("/nonexistent/path/to/file.raw")


def test_verify_hash_correct():
    """verify_hash returns True for an unmodified file."""
    with tempfile.NamedTemporaryFile(delete=False) as f:
        f.write(b"unchanged content")
        tmp_path = f.name

    try:
        sha256, _ = compute_sha256(tmp_path)
        assert verify_hash(tmp_path, sha256) is True
```

```
    finally:
        os.unlink(tmp_path)


def test_verify_hash_tampered():
    """verify_hash returns False if file content change
s."""
    with tempfile.NamedTemporaryFile(delete=False) as f:
        f.write(b"original content")
        tmp_path = f.name

    try:
        sha256, _ = compute_sha256(tmp_path)
        with open(tmp_path, "wb") as f:
            f.write(b"tampered content")
        assert verify_hash(tmp_path, sha256) is False
    finally:
        os.unlink(tmp_path)


# ── ARTIFACT TESTS ──────────────────────────────────────
──────────────────────


def test_artifact_creation():
    """Artifact is created with correct fields."""
    a = Artifact(
        case_id="TEST-001",
        source_module="memory",
        artifact_type="process",
        host_id="192.168.1.10",
        data={"pid": 1234, "name": "malware.exe", "ppid": 5
00},
        timestamp="2026-05-03T10:00:00+00:00"
    )
    assert a.case_id == "TEST-001"
    assert a.source_module == "memory"
    assert a.data["pid"] == 1234
```

```
def test_artifact_invalid_module():
    """Artifact raises ValueError for invalid source_modul
e."""
    with pytest.raises(ValueError):
        Artifact(
            case_id="TEST-001",
            source_module="invalid_module",
            artifact_type="process",
            host_id="192.168.1.10",
            data={}
        )


def test_artifact_serialisation():
    """Artifact serialises to dict and back correctly."""
    a = Artifact(
        case_id="TEST-001",
        source_module="network",
        artifact_type="dns_query",
        host_id="10.0.0.5",
        data={"query": "malicious.com", "response": "1.2.3.
4"},
        timestamp="2026-05-03T10:00:00+00:00"
    )
    d = a.to_dict()
    a2 = Artifact.from_dict(d)
    assert a2.case_id == a.case_id
    assert a2.data["query"] == "malicious.com"


def test_artifact_summary():
    """Artifact summary returns a non-empty string."""
    a = Artifact(
        case_id="TEST-001",
        source_module="logs",
        artifact_type="failed_login",
        host_id="server-01",
```

```
        data={"user": "root", "ip": "10.0.0.99", "attempt
s": 5}
    )
    summary = a.summary()
    assert isinstance(summary, str)
    assert len(summary) > 0
    assert "LOGS" in summary
```



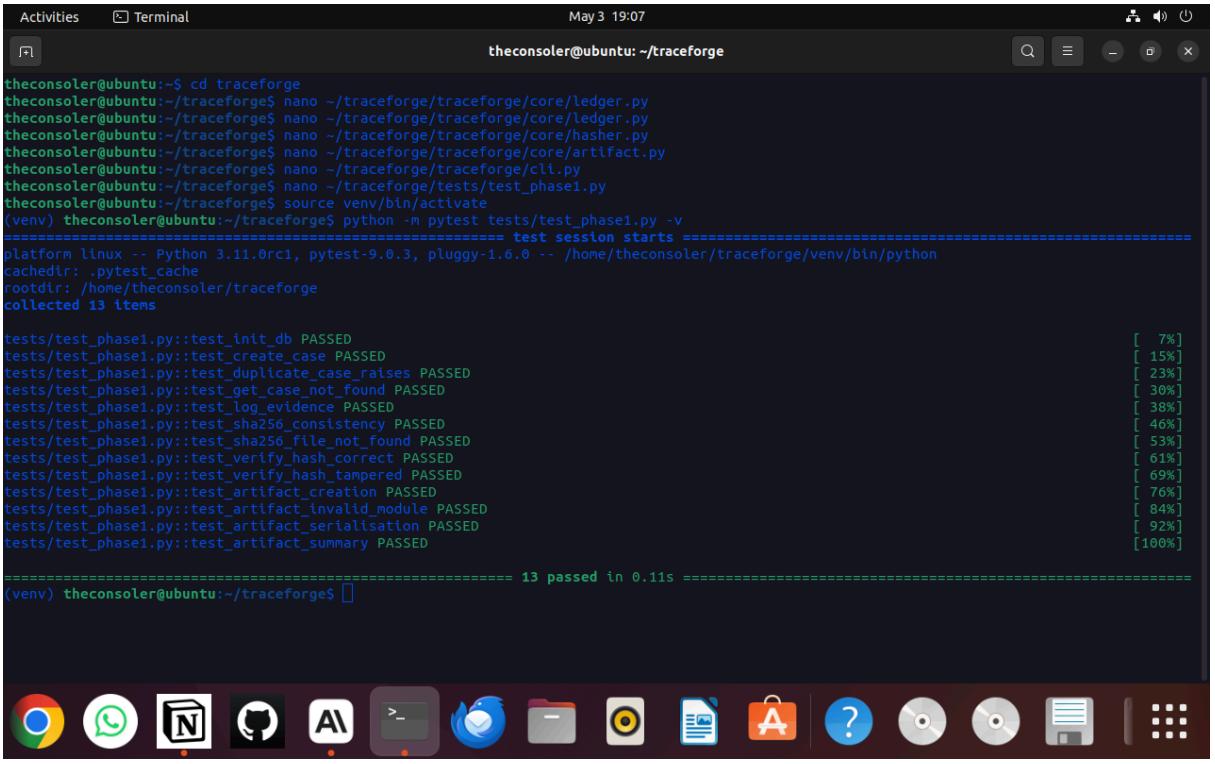

# Step 1.7 — Run the tests

```
cd ~/traceforge
source venv/bin/activate
python -m pytest tests/test_phase1.py -v
```

**Expected output:**

```
tests/test_phase1.py::test_init_db                    PASSED
tests/test_phase1.py::test_create_case                PASSED
tests/test_phase1.py::test_duplicate_case_raises      PASSED
tests/test_phase1.py::test_get_case_not_found         PASSED
tests/test_phase1.py::test_log_evidence               PASSED
tests/test_phase1.py::test_sha256_consistency         PASSED
tests/test_phase1.py::test_sha256_file_not_found      PASSED
```

```
tests/test_phase1.py::test_verify_hash_correct          PASSED
tests/test_phase1.py::test_verify_hash_tampered         PASSED
tests/test_phase1.py::test_artifact_creation            PASSED
tests/test_phase1.py::test_artifact_invalid_module      PASSED
tests/test_phase1.py::test_artifact_serialisation       PASSED
tests/test_phase1.py::test_artifact_summary             PASSED

13 passed in X.XXs
```




**If any test fails:** Do not move to Phase 2. Fix the failing test first.

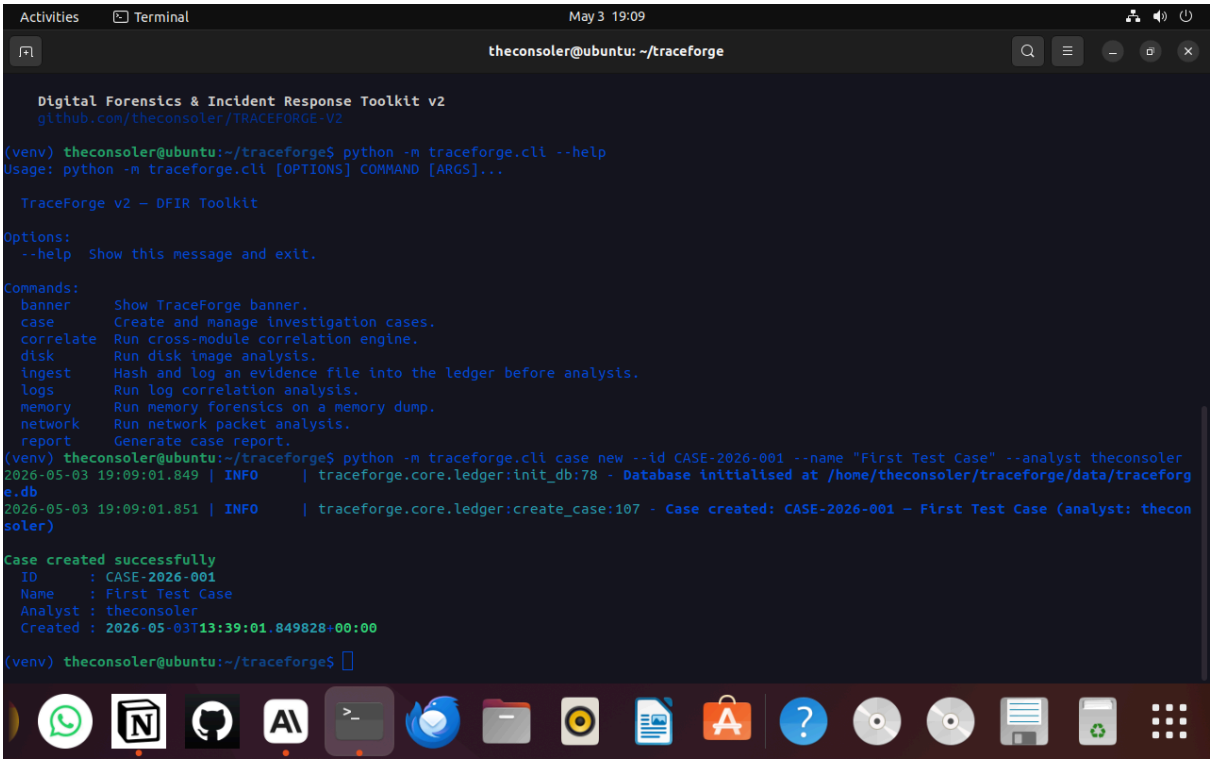
---

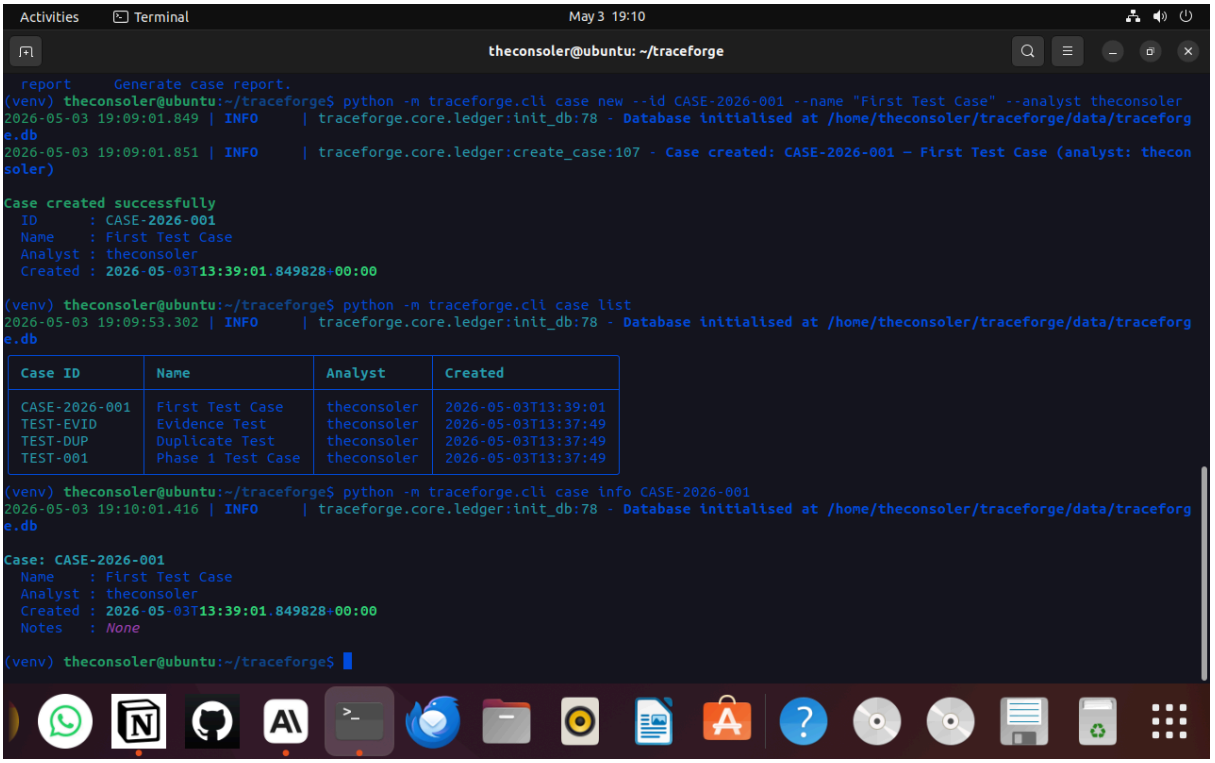
## Step 1.8 — Test the CLI manually

```
cd ~/traceforge
source venv/bin/activate
python -m traceforge.cli banner
python -m traceforge.cli --help
python -m traceforge.cli case new --id CASE-2026-001 --name
"First Test Case" --analyst theconsoler
python -m traceforge.cli case list
python -m traceforge.cli case info CASE-2026-001
```

The banner should print. The case should be created, listed, and retrievable.

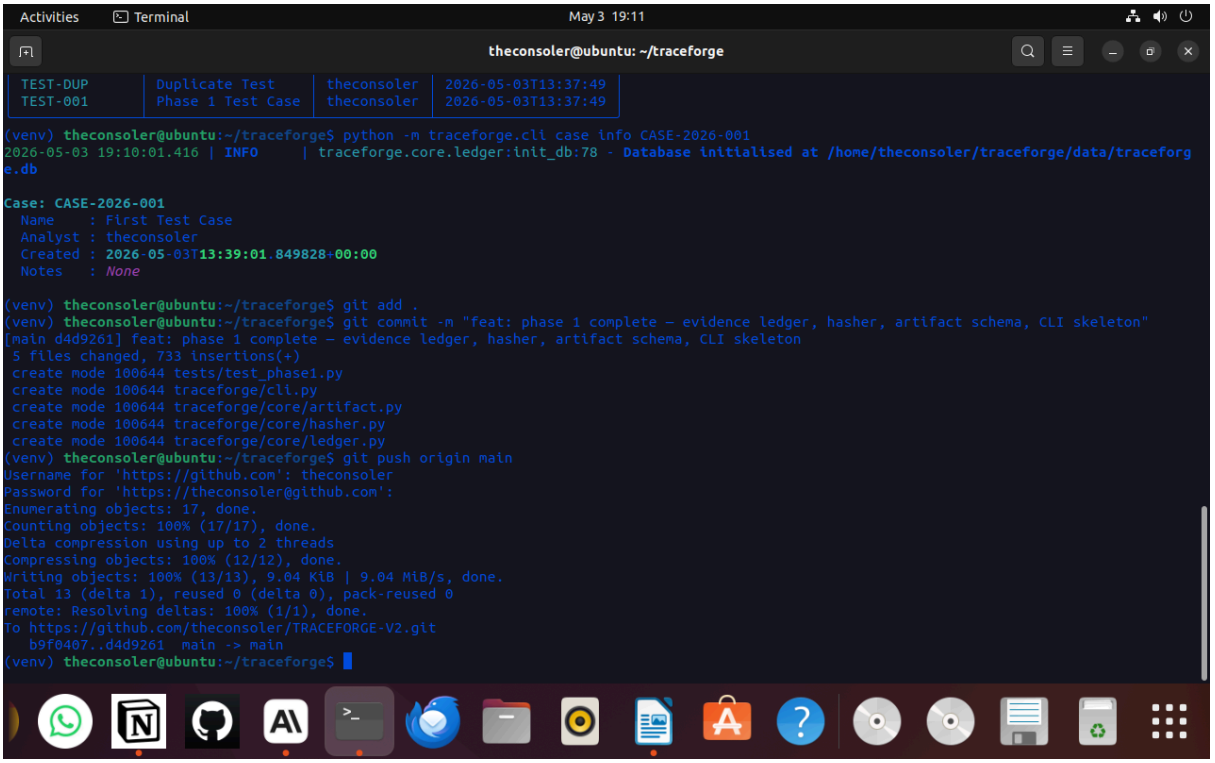

Activities
Terminal
May 3 19:08
theconsoler@ubuntu: ~/traceforge
========== 13 passed in 0.11s ==========
(venv) theconsoler@ubuntu:~/traceforge$ python -m traceforge.cli banner
2026-05-03 19:08:22.950 | INFO | traceforge.core.ledger:init_db:78 - Database initialised at /home/theconsoler/traceforge/data/traceforge.db
TRACEFORGE
Digital Forensics & Incident Response Toolkit v2
github.com/theconsoler/TRACEFORGE-V2
(venv) theconsoler@ubuntu:~/traceforge$ python -m traceforge.cli --help
Usage: python -m traceforge.cli [OPTIONS] COMMAND [ARGS]...
TraceForge v2 — DFIR Toolkit
Options:
--help Show this message and exit.
Commands:
banner Show TraceForge banner.
case Create and manage investigation cases.
correlate Run cross-module correlation engine.
disk Run disk image analysis.
ingest Hash and log an evidence file into the ledger before analysis.
logs Run log correlation analysis.
memory Run memory forensics on a memory dump.
network Run network packet analysis.
report Generate case report.
(venv) theconsoler@ubuntu:~/traceforge$


Activities
Terminal
May 3 19:09
theconsoler@ubuntu: ~/traceforge
Digital Forensics & Incident Response Toolkit v2
github.com/theconsoler/TRACEFORGE-V2
(venv) theconsoler@ubuntu:~/traceforge$ python -m traceforge.cli --help
Usage: python -m traceforge.cli [OPTIONS] COMMAND [ARGS]...
TraceForge v2 — DFIR Toolkit
Options:
--help Show this message and exit.
Commands:
banner Show TraceForge banner.
case Create and manage investigation cases.
correlate Run cross-module correlation engine.
disk Run disk image analysis.
ingest Hash and log an evidence file into the ledger before analysis.
logs Run log correlation analysis.
memory Run memory forensics on a memory dump.
network Run network packet analysis.
report Generate case report.
(venv) theconsoler@ubuntu:~/traceforge$ python -m traceforge.cli case new --id CASE-2026-001 --name "First Test Case" --analyst theconsoler
2026-05-03 19:09:01.849 | INFO | traceforge.core.ledger:init_db:78 - Database initialised at /home/theconsoler/traceforge/data/traceforge.db
2026-05-03 19:09:01.851 | INFO | traceforge.core.ledger:create_case:107 - Case created: CASE-2026-001 — First Test Case (analyst: theconsoler)
Case created successfully
ID : CASE-2026-001
Name : First Test Case
Analyst : theconsoler
Created : 2026-05-03T13:39:01.849828+00:00
(venv) theconsoler@ubuntu:~/traceforge$




## Step 1.9 — Commit and push Phase 1

```
cd ~/traceforge
git add .
git commit -m "feat: phase 1 complete — evidence ledger, hasher, artifact schema, CLI skeleton"
git push origin main
```




# Phase 1 completion checklist

- [x] `traceforge/core/ledger.py` ~~created and working~~
- [x] `traceforge/core/hasher.py` ~~created and working~~
- [x] `traceforge/core/artifact.py` ~~created and working~~
- [x] `traceforge/cli.py` ~~created with all subcommand stubs~~
- [x] `data/` ~~directory created~~
- [x] ~~All 13 pytest tests pass~~
- [x] ~~CLI banner prints without errors~~
- [x] `traceforge case new` ~~creates a case successfully~~
- [x] ~~Phase 1 committed and pushed to GitHub~~

**When every box is checked: Phase 1 is complete. Move to Phase 2.**

---

*TraceForge v2 build documentation — Phase 1 of 4Last updated: May 2026*